

1. 核心思想 (Core Idea)

加权平均法 (Weighted Average Method) 是一种用于将模糊集合转化为具体数值（即“去模糊化”）的方法。其中模糊集A（即模糊推理的输出）是M个模糊集的并集。

它适用于输出隶属函数对称且分布较规则的模糊系统。

主要思想：

- 对每个输出模糊集（即语言变量对应的值）取其**最大隶属度 $\mu(x)$** ；
- 使用这些最大隶属度作为权重，对对应的清晰值 x 进行加权平均；
- 计算公式如下：

$$x^* = \frac{\sum [\mu(x_i) \times x_i]}{\sum [\mu(x_i)]}$$

其中：

- x_i : 每个语言变量对应的清晰值（例如：60, 70, 80...）
- $\mu(x_i)$: 该值对应的最大隶属度

这种方法的优点是计算简单、结果与 COA（重心法）非常接近，但计算量更小。

2. 示例说明 (Example)

用一个例子展示 **加权平均法 (Weighted Average Method)** 的计算思路。

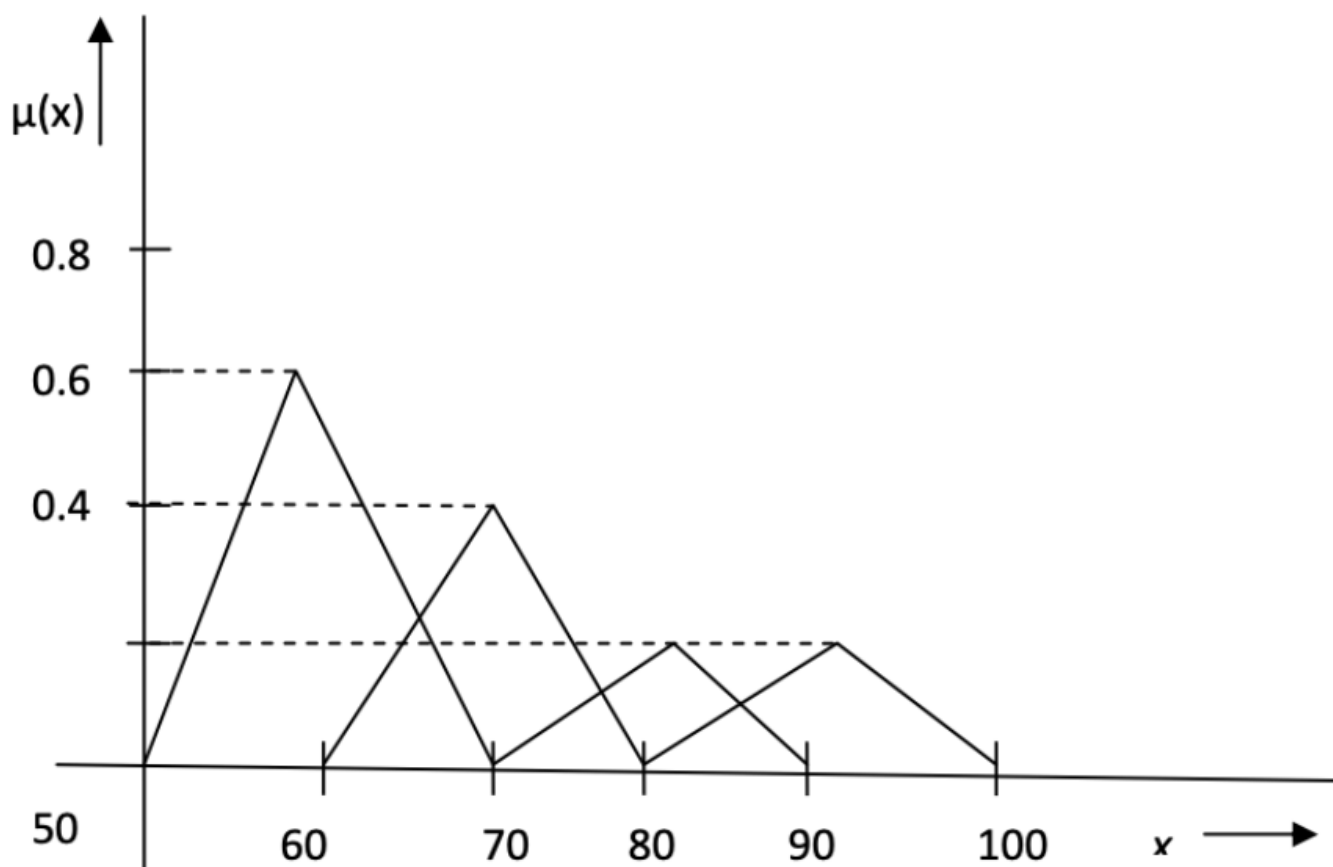
所以它并不是一个完整的“模糊控制系统”，而是一个“输出层的例子”——已经得到了几个模糊输出（不同成绩等级及其隶属度），现在要把这些模糊结果转化为一个**明确的成绩分数（清晰值 crisp output）**。

设模糊集合 A 表示“学生成绩评价”：

Linguistic term	Meaning	x	$\mu(x)$
P	Pass	60	0.6
F	Fair	70	0.4
G	Good	80	0.2
VG	Very Good	90	0.2

Linguistic term	Meaning	x	$\mu(x)$
E	Excellent	100	0.0

fuzzy set A



$$A = \{(P, 0.6), (F, 0.4), (G, 0.2), (VG, 0.2), (E, 0)\}$$

我们把学生成绩的输出模糊集设计得“规则、对称、均匀”，是为了让去模糊化过程更直观、易理解，而不是完全还原真实的复杂情况。

接下来就是主要介绍weighted average method的计算。

根据公式：

$$x^* = \frac{\sum [\mu(x_i) \times x_i]}{\sum [\mu(x_i)]}$$

$$x^* = \frac{60 \times 0.6 + 70 \times 0.4 + 80 \times 0.2 + 90 \times 0.2 + 100 \times 0}{0.6 + 0.4 + 0.2 + 0.2 + 0}$$

$$x^* = \frac{98}{1.4} = 70$$

因此，学生的综合成绩对应的去模糊化结果为 **70 (Fair)**。

3. Python 实现

下面是基于该例子的 Python 实现，包括计算与可视化图表。

```
import numpy as np
import matplotlib.pyplot as plt

def triangular(x, a, b, c, height=1.0):
    """
    带高度参数的三角形隶属函数
    a: 左端点
    b: 顶点
    c: 右端点
    height: 顶点处的隶属度（允许 < 1）
    """
    # 计算标准三角形（顶点高度为1）的值
    # 为避免除以0，处理退化区间
    left = np.where(b - a > 0, (x - a) / (b - a), 0.0)
    right = np.where(c - b > 0, (c - x) / (c - b), 0.0)
    mu_unit = np.maximum(np.minimum(left, right), 0.0)
    return height * mu_unit

# ----- x 网格 -----
x = np.linspace(50, 100, 1000)

# ----- 定义每个语言变量的三角形 -----
# 顶点位置: 60, 70, 80, 90, 100
# 对应的最大隶属度: 0.6, 0.4, 0.2, 0.2, 0.0
peaks = [60, 70, 80, 90, 100]
heights = [0.6, 0.4, 0.2, 0.2, 0.0]

# 为每个三角形选择左右端点（每个三角宽度为20）
mus = []
for i, p in enumerate(peaks):
    a = p - 10    # 左端
    b = p         # 顶点
    c = p + 10    # 右端
```

```

# 对于最右侧 E (peak at 100) 让右端不超过100
if c > 100:
    c = 100
mu = triangular(x, a, b, c, height=heights[i])
mus.append(mu)

mu_P, mu_F, mu_G, mu_VG, mu_E = mus

# ----- 从图形中提取每个模糊子集的最大隶属度及其对应 x (自动) -----
mu_values = [np.max(mu) for mu in mus]
x_values = [x[np.argmax(mu)] if np.max(mu) > 0 else peaks[i] for i, mu in
enumerate(mus)]
# 上面对 mu==0 的情况 (如Excellent) 用峰值位置作为 x 值 (但权重为0, 不影响计算)

# ----- 计算加权平均去模糊化 -----
mu_sum = np.sum(mu_values)
if mu_sum == 0:
    x_star = np.nan
else:
    x_star = np.sum(np.array(x_values) * np.array(mu_values)) / mu_sum

# ----- 输出数值 -----
print("Peaks (x):", peaks)
print("Heights ( $\mu$  peaks):", heights)
print("Extracted x_values:", [round(v,2) for v in x_values])
print("Extracted mu_values:", [round(v,3) for v in mu_values])
print(f"Defuzzified value (x*): {x_star:.2f}")

# ----- 绘图 -----
plt.figure(figsize=(9,5))
plt.plot(x, mu_P, label='P (Pass)', linewidth=2)
plt.plot(x, mu_F, label='F (Fair)', linewidth=2)
plt.plot(x, mu_G, label='G (Good)', linewidth=2)
plt.plot(x, mu_VG, label='VG (Very Good)', linewidth=2)
plt.plot(x, mu_E, label='E (Excellent)', linewidth=2)

# 标出每个子集的顶点和最大隶属度的虚线
for xi, mui, lab in zip(x_values, mu_values, ['P', 'F', 'G', 'VG', 'E']):
    if mui > 0:
        plt.plot(xi, mui, 'ko') # 标点
        plt.hlines(mui, 50, xi, colors='gray', linestyle='dashed',
linewidth=1)
        plt.vlines(xi, 0, mui, colors='gray', linestyle='dotted',
linewidth=1)

# 去模糊化结果线

```

```

if not np.isnan(x_star):
    plt.axvline(x_star, color='red', linestyle='--', linewidth=1.5,
                label=f'Defuzzified x* = {x_star:.2f}')

plt.xlim(50, 100)
plt.ylim(-0.02, 0.72)
plt.xlabel('x (Score)')
plt.ylabel('Membership degree  $\mu(x)$ ')
plt.title('Fuzzy sets (triangles) and Weighted Average defuzzification')
plt.grid(True, linestyle=':', linewidth=0.5)
plt.legend()
plt.show()

```



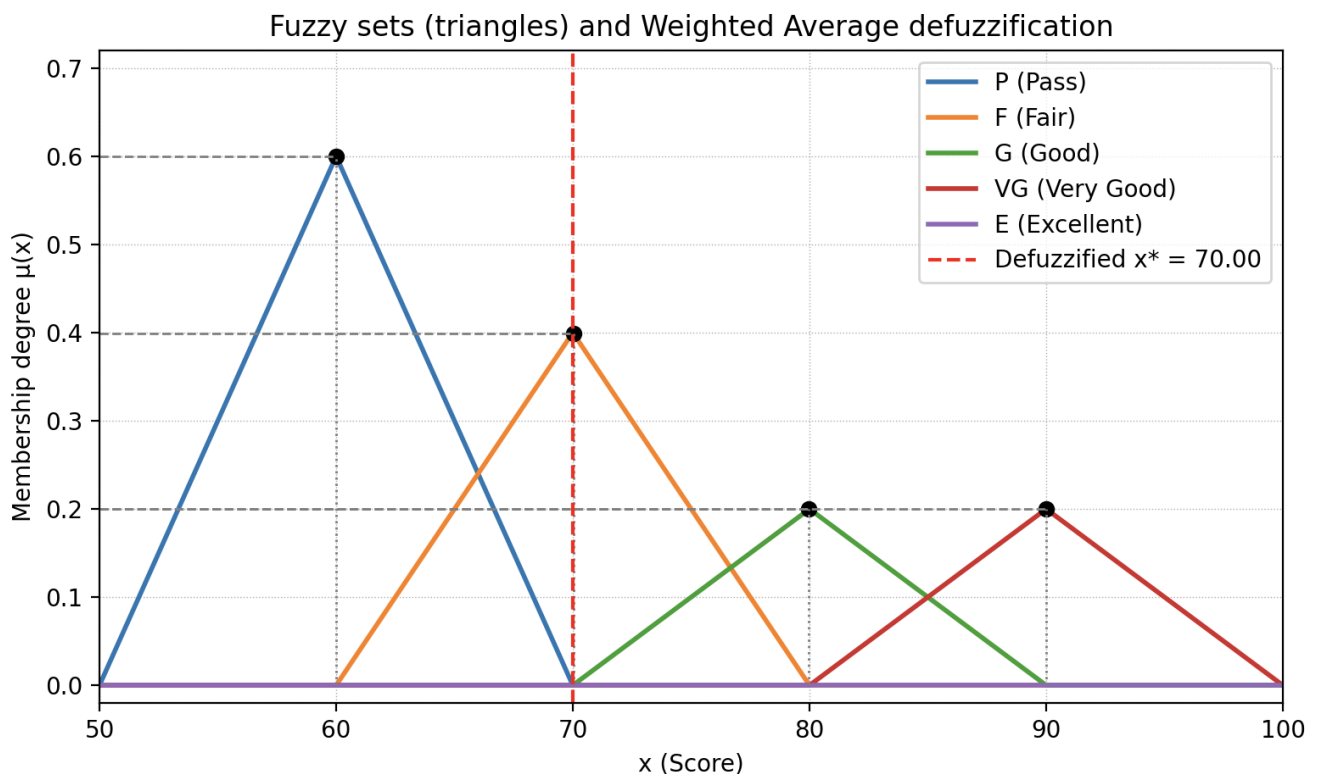
4. 结果展示与分析

输出结果

Defuzzified value (x^*): 70.0

图形将展示：

- 5个三角形隶属函数组成的模糊集；
- 每个语言变量的最大隶属度虚线；
- - 黑点与灰色虚线标出每个子集的最大隶属度和对应 x ；
- 红色虚线显示最终去模糊化结果 $x^*=70$ 。



5. 小结

项目	内容
方法名称	Weighted Average Method
适用场景	对称、规则的输出隶属函数
优点	计算简单、结果接近 COA
缺点	对非对称函数不太精确
示例结果	$(x^* = 70)$ (Fair student)